**Prakash Thapa**

**D20A -32**

## Experiment No. 3

## Create a Cryptocurrency using Python and perform mining in the Blockchain created.

### Aim

To create a cryptocurrency using Python and perform mining in a blockchain using a simulated three-node Peer-to-Peer (P2P) network.

### Theory:

1. Blockchain

A blockchain is a distributed and decentralized ledger in which information is stored in a sequence of linked blocks. Each block contains transaction data, a timestamp, a proof value, and the hash of the previous block. The previous hash connects the blocks and helps detect unauthorized changes.

2. Cryptocurrency and Transactions

A cryptocurrency is represented in this experiment through transactions containing a sender, receiver, and amount. Transactions are initially stored in a temporary collection called the mempool. When a miner mines a block, the pending transactions are added to that block.

3. Mining

Mining is the process of collecting pending transactions and creating a valid block using Proof-of-Work (PoW). The miner searches for a valid proof that satisfies the cryptographic condition. After successful mining, a reward transaction is added to the block for the miner.

4. P2P Blockchain Network

The experiment uses three independent blockchain nodes running on ports 5001, 5002, and 5003. Each node maintains its own copy of the blockchain and communicates with other nodes using HTTP requests through Flask and the Requests library.

5. Longest Chain Rule

When different nodes have different versions of the blockchain, the longest valid chain is selected. The /replace_chain operation allows a node to obtain the chains of its peers and replace its own chain if a longer valid chain is found.

6. Mempool Modification

Pending transactions are stored in the mempool before mining. After the transactions are added to a successfully mined block, they are removed from the pending transaction list. This prevents the same transactions from being added repeatedly to another block.

**Tools and Libraries Used**

- Python 3.x – Programming language used for implementation.
- Flask – Used to create the blockchain REST API and run the three nodes.
- Requests – Used for HTTP communication between blockchain nodes.
- Postman – Used to send and test API requests.
- datetime – Used to generate timestamps.
- hashlib – Used to generate SHA-256 hashes and implement Proof-of-Work.
- json – Used for encoding block data.
- uuid – Used to generate a unique identifier for each node.
- urllib.parse – Used for parsing peer node URLs.

**Procedure**

1. Created three Python files:
    - hadcoin_node_5001.py
    - hadcoin_node_5002.py
    - hadcoin_node_5003.py

Installed the required Python packages:
pip install Flask
pip install requests

2. Started the three blockchain nodes in separate terminals:

python hadcoin_node_5001.py

python hadcoin_node_5002.py

python hadcoin_node_5003.py

3. Connected the three nodes using the /connect_node API through Postman.
4. Added three transactions to Node 5001 using the /add_transaction API.
5. Mined a block on Node 5001 using /mine_block. The pending transactions were added to the block along with the mining reward.
6. Verified that the transactions were removed from the pending transaction list after mining.
7. Retrieved the blockchain using /get_chain.
8. Used /replace_chain on Nodes 5002 and 5003 to obtain the longest valid blockchain from their peers.
9. Verified that all three nodes contained the synchronized blockchain.
10. Used /is_valid to verify the validity of the blockchain.

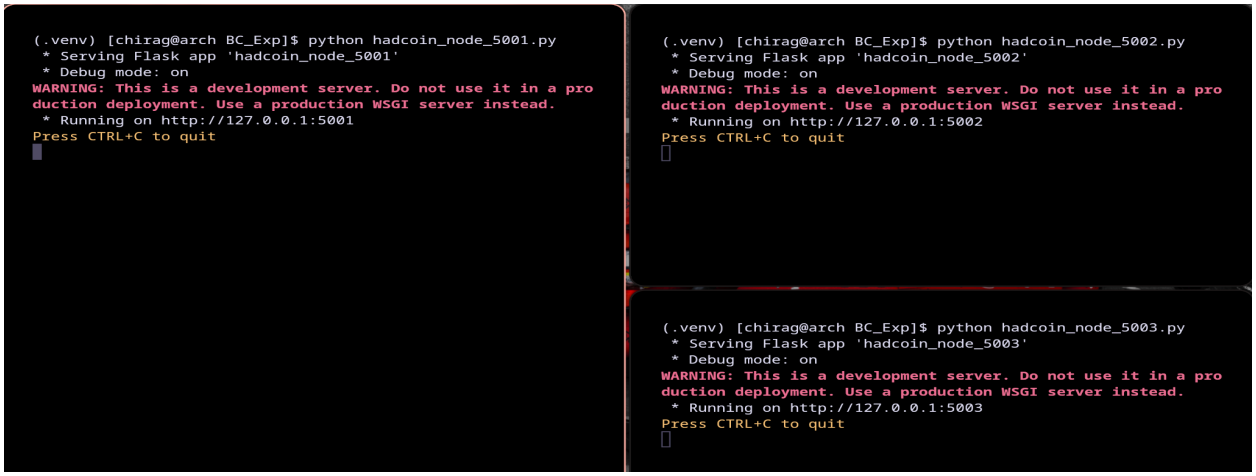
**API Endpoints Used**

| Method | Endpoint | Purpose |
|---|---|---|
| POST | /connect_node | Connect blockchain nodes |
| POST | /add_transaction | Add a cryptocurrency transaction |
| GET | /mine_block | Mine a new block |
| GET | /get_chain | Display the blockchain |
| GET | /replace_chain | Replace the current chain with the longest valid chain |
| GET | /is_valid | Verify blockchain validity |

**Output Description**

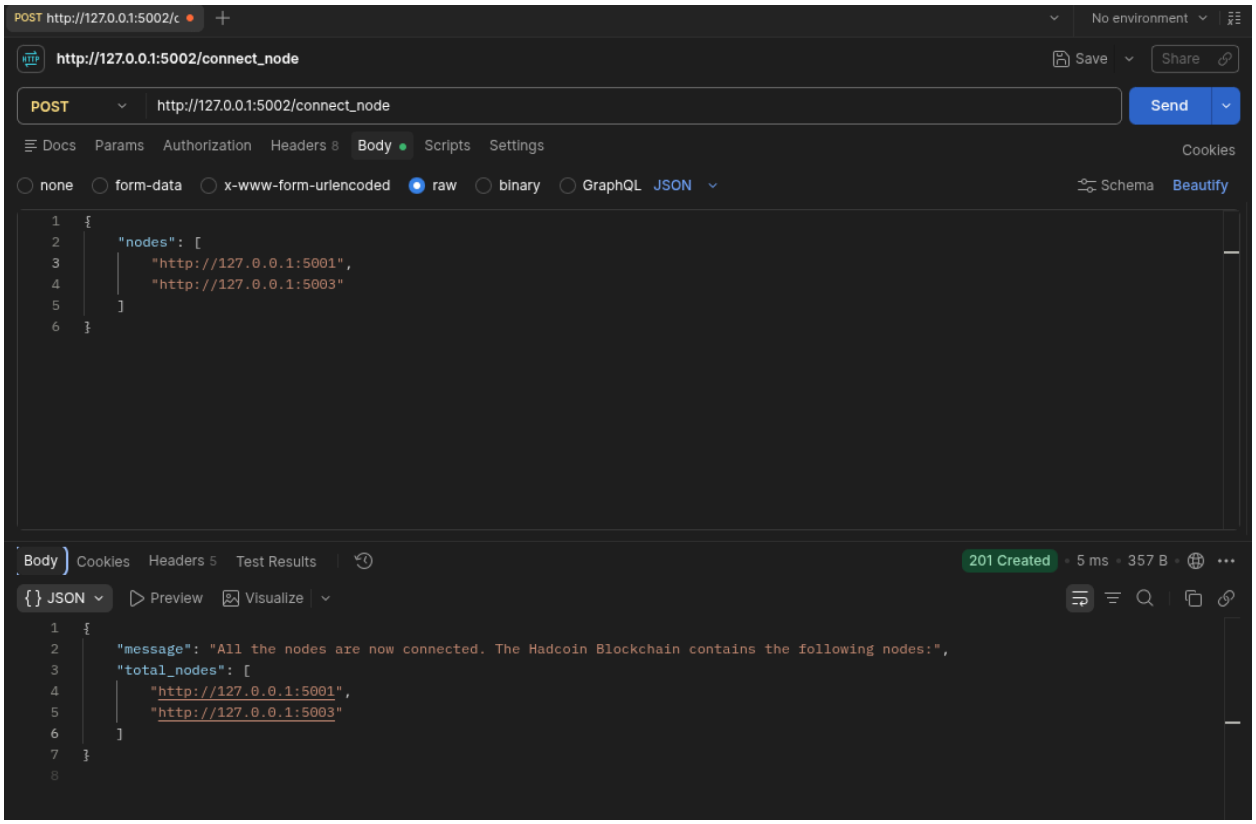
Output 1 – Three Nodes Running

Three Flask servers were successfully started on ports 5001, 5002, and 5003, representing three independent blockchain nodes.



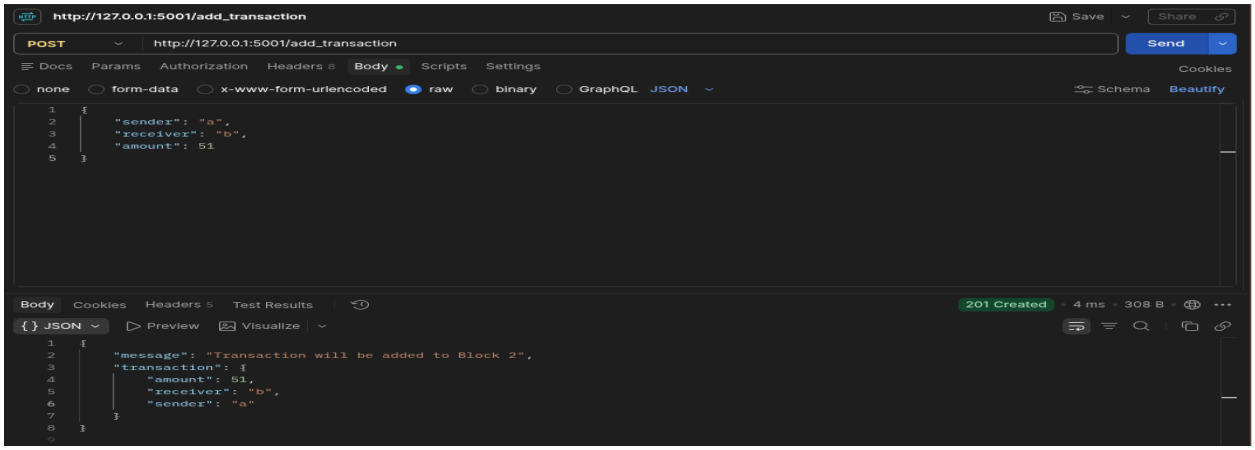

Output 2 – Nodes Connected

The /connect_node API successfully connected the three nodes and established the simulated P2P network.



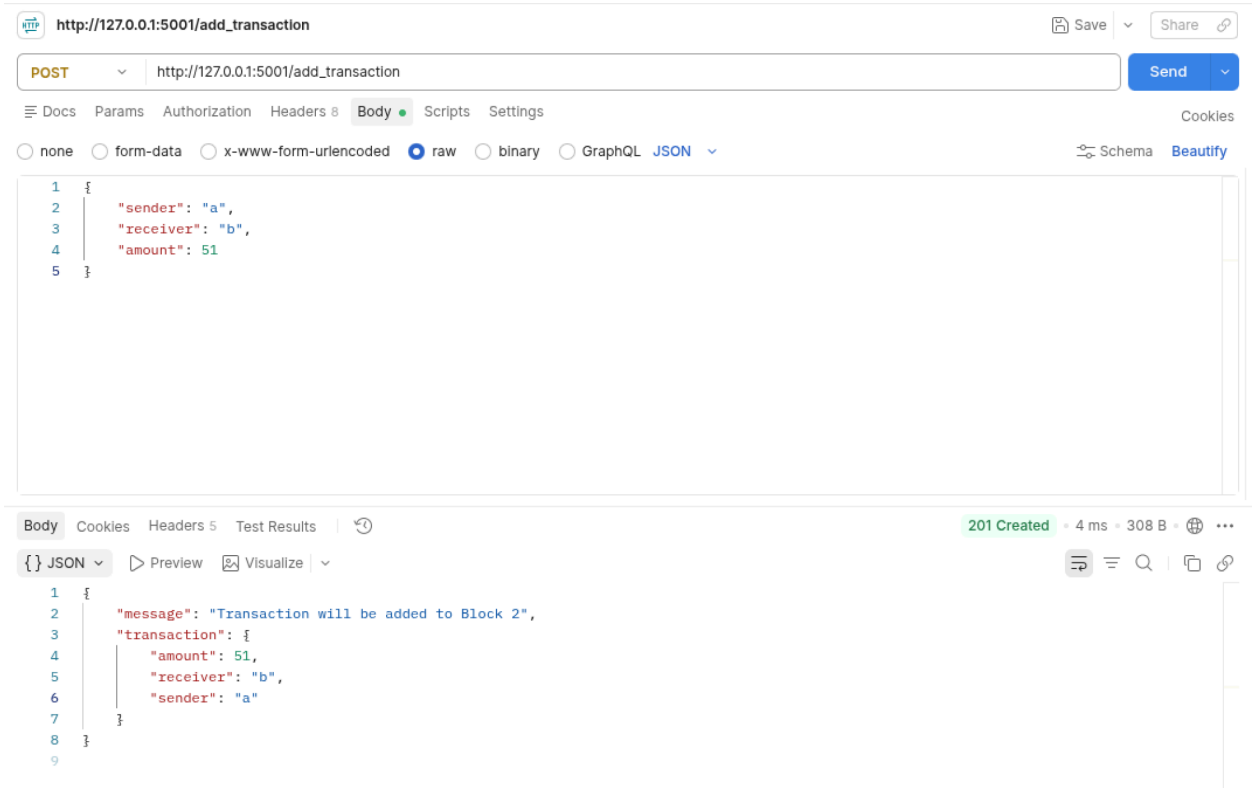

Output 3 – Transactions Added

Three transactions were successfully added to Node 5001. The transactions remained in the node's pending transaction list until mining.



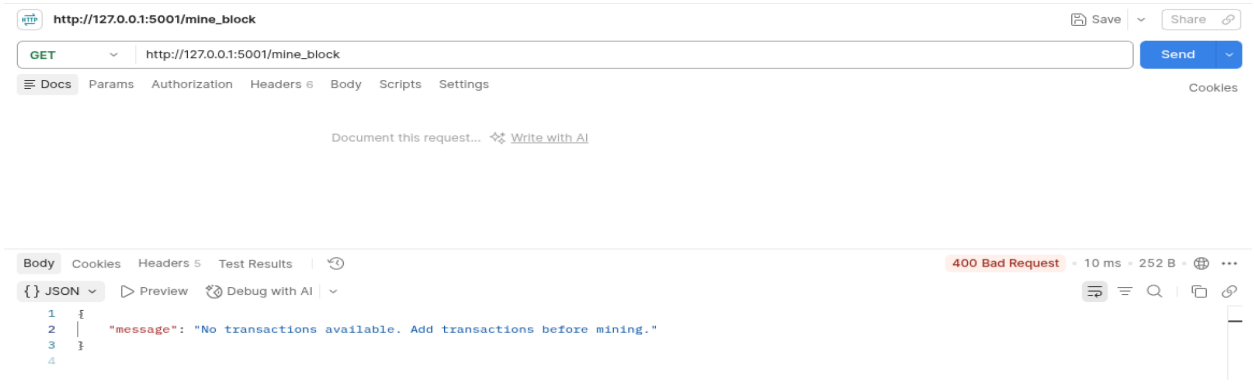

Output 4 – Block Mined

Node 5001 successfully mined a block containing the pending transactions and a mining reward. The response displayed the block index, timestamp, proof, previous hash, and transactions.



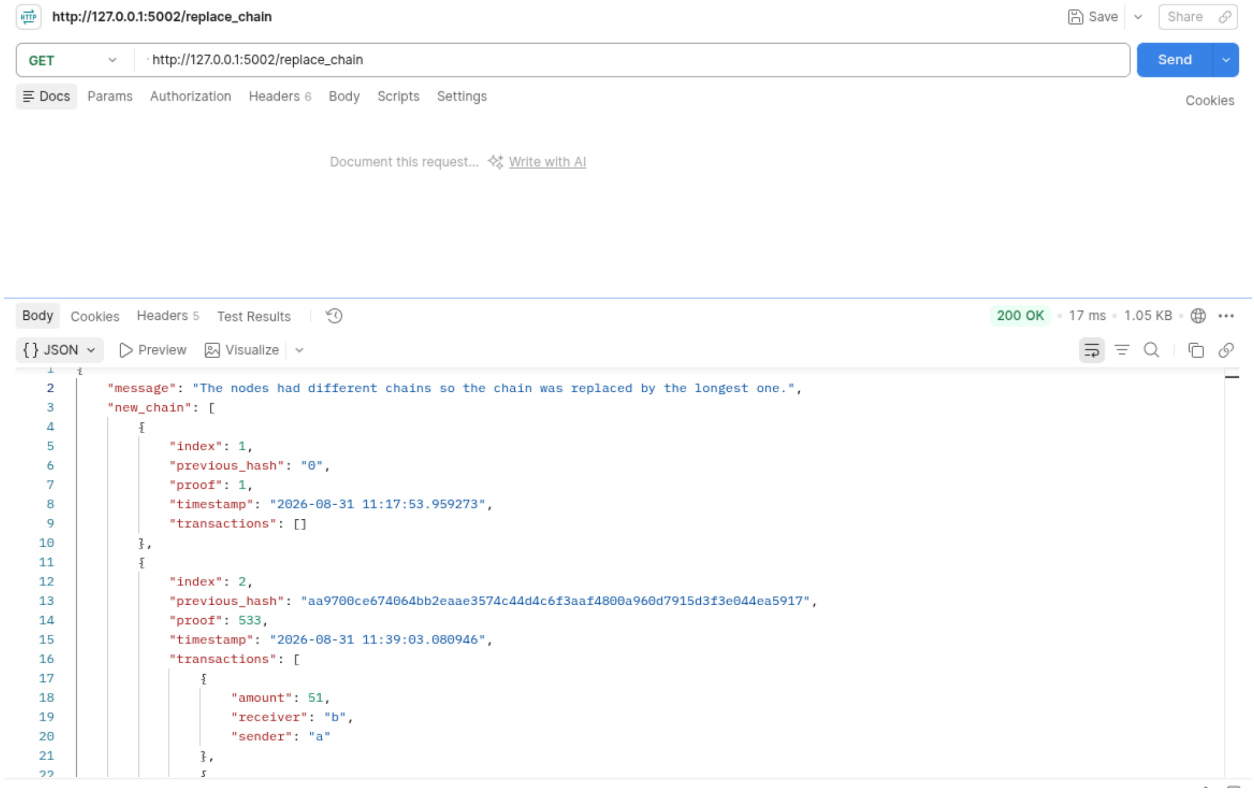

Output 5 – Transactions Removed

After mining, the transactions were removed from the pending transaction list, confirming the required modification.



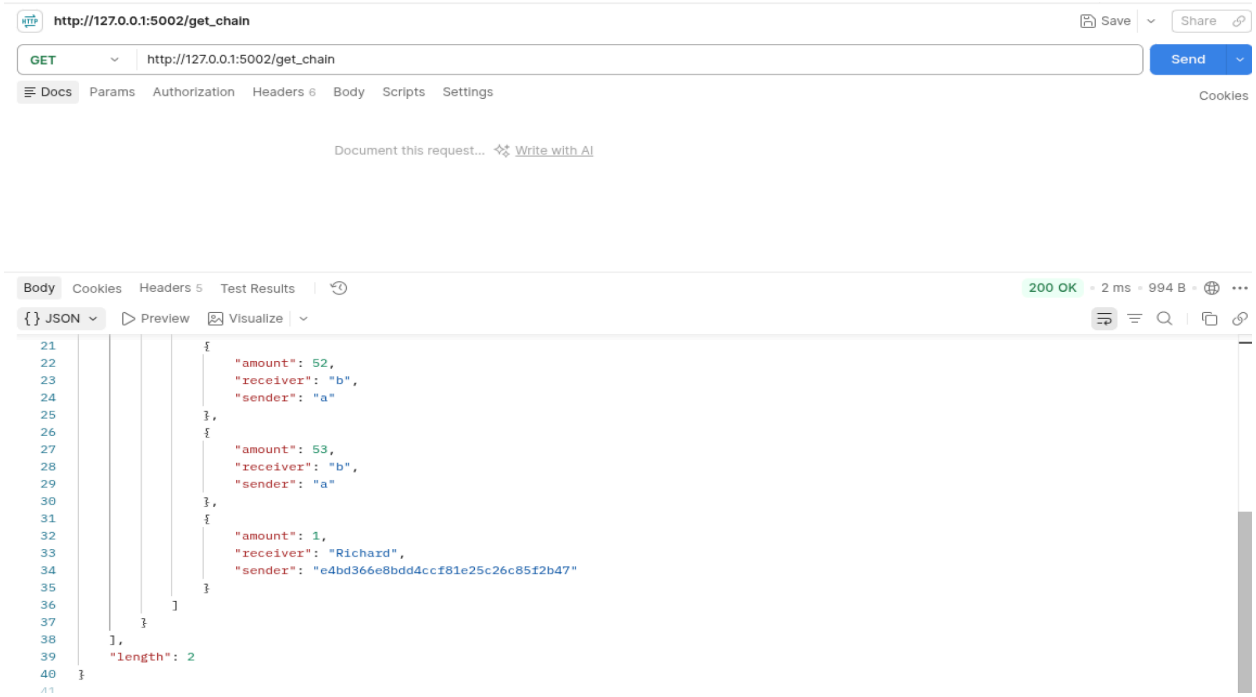

Output 6 – Chain Replacement

Initially, the nodes had different chain lengths. Nodes 5002 and 5003 used /replace_chain to obtain the longer valid chain from their peers.



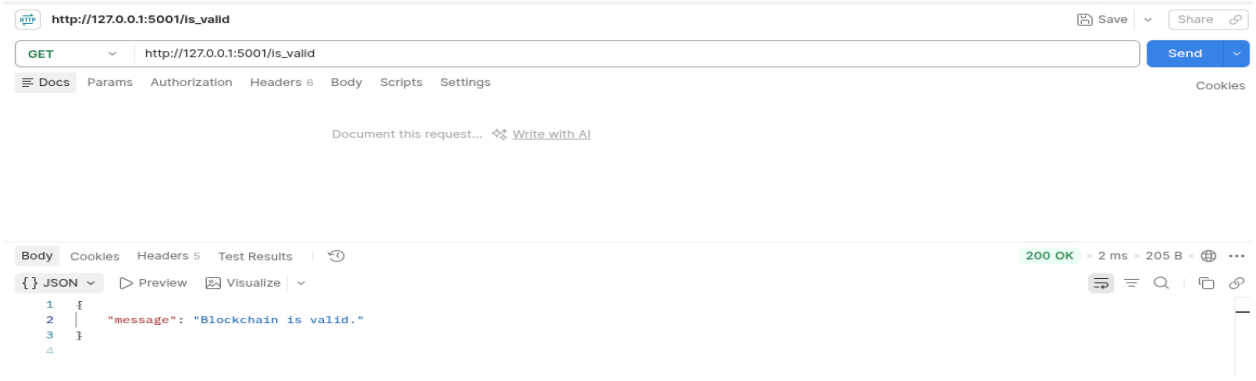

Output 7 – Blockchain Synchronization

After chain replacement, all three nodes contained the same longest valid blockchain.




Output 8 – Blockchain Validation

The /is_valid endpoint confirmed that the blockchain was valid.




## Conclusion

The cryptocurrency was successfully implemented in Python using Flask, Postman, and a simulated three-node P2P blockchain network. Transactions were added to the mempool and successfully included in mined blocks using Proof-of-Work. The Longest Chain Rule was used to synchronize different blockchain copies among the nodes. The required modification was also

implemented so that transactions were removed from the pending transaction list after being added to a block. Thus, the experiment demonstrated cryptocurrency transactions, mining, P2P communication, blockchain synchronization, and blockchain validation.